

Multi-Region Architecture Design

Executive Summary

This document outlines the design for deploying the Presentation Coaching Platform across two AWS regions in an active-active configuration with geographic routing and user session stickiness. The architecture enables low-latency access for geographically distributed users, provides high availability through automated failover, and maintains data consistency across regions.

1. Region Selection

Recommended: us-east-1 (N. Virginia) + us-west-2 (Oregon)

Factor	us-east-1	us-west-2
Bedrock Model Availability	Full (all models incl. Nova Pro, Claude)	Full (all models incl. Nova Pro, Claude)
Amazon Transcribe	✓	✓
Cognito	✓	✓
Step Functions	✓	✓
DynamoDB Global Tables	✓	✓

Factor	us-east-1	us-west-2
S3 Cross-Region Replication	✓	✓
Geographic Coverage	US East Coast, Europe (closer)	US West Coast, Asia-Pacific (closer)
Cost	Standard pricing	Standard pricing
Latency from East Coast	~10-30ms	~60-80ms
Latency from West Coast	~60-80ms	~10-30ms

Why This Pairing

- **Fault isolation:** Separate availability zone groups, separate power grids, separate network paths
- **Bedrock parity:** Both regions support Claude Sonnet, Nova Pro, and Nova Embeddings
- **Geographic spread:** Covers continental US with reasonable latency for both coasts
- **Service maturity:** Both are Tier 1 regions with full service availability
- **Cost neutral:** No pricing difference between these regions for the services used

Alternative Considered: us-east-1 + eu-west-1

Better for European users but introduces GDPR data residency complexity. Consider this pairing if your user base is significantly European.

2. Architecture Overview



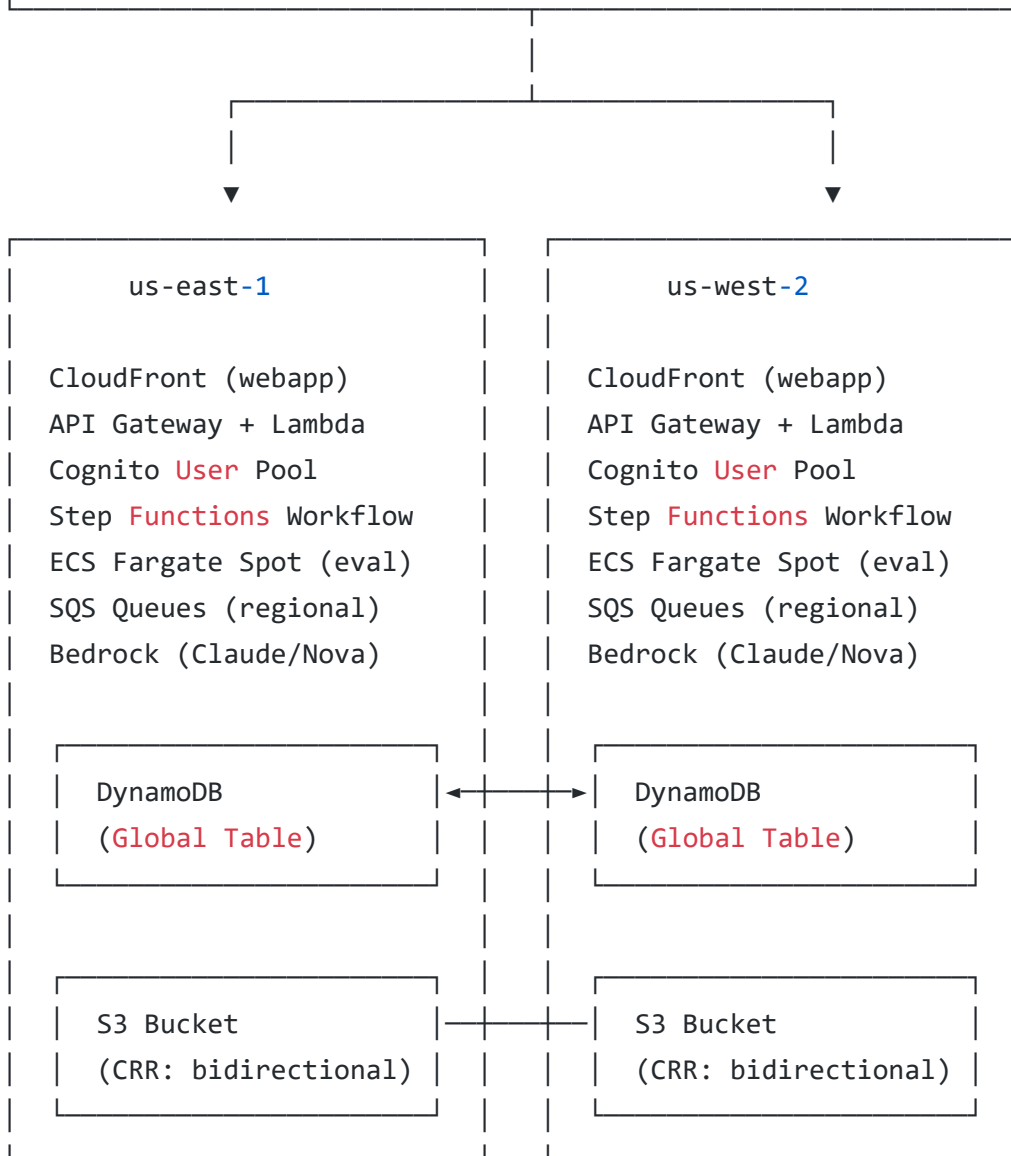
api.prescoach.geiserai.com → Geolocation Routing Policy
kiro.geiserai.com (webapp) → Geolocation Routing Policy

Health Checks:

- └ us-east-1 health endpoint (/health)
- └ us-west-2 health endpoint (/health)

Routing Logic:

- └ US-East users → us-east-1 (primary)
- └ US-West users → us-west-2 (primary)
- └ Default/Other → us-east-1
- └ Failover: unhealthy region → healthy region



3. Route 53 Configuration

3.1 Routing Strategy: Geolocation with Health Check Failover

Use **Geolocation Routing** (not Latency-based) combined with health checks to achieve:

- Geographic affinity (users route to nearest region)
- Sticky sessions (same user consistently hits the same region)
- Automatic failover when a region is unhealthy

Route 53 Records:

`api.prescoach.geiserai.com`

- └ Type: A (Alias)
- └ Routing: Geolocation
- └ Record 1: Location=North America/US-East → us-east-1 API GW
 - Health Check: us-east-1-api-health
 - Failover: if unhealthy → us-west-2
- └ Record 2: Location=North America/US-West → us-west-2 API GW
 - Health Check: us-west-2-api-health
 - Failover: if unhealthy → us-east-1
- └ Record 3: Location=Default → us-east-1 API GW
 - Health Check: us-east-1-api-health

3.2 Health Check Design

Each region exposes a `/health` endpoint via API Gateway + Lambda that validates:

```
# Health check Lambda – validates all critical dependencies
def handler(event, context):
    checks = {}

    # 1. DynamoDB reachable
    try:
        dynamodb.describe_table(TableName=TABLE_NAME)
        checks["dynamodb"] = "healthy"
    except Exception as e:
        checks["dynamodb"] = f"unhealthy: {e}"
```

```

# 2. SQS queue accessible
try:
    sqs.get_queue_attributes(QueueUrl=QUEUE_URL, AttributeNames=["All"])
    checks["sqs"] = "healthy"
except Exception as e:
    checks["sqs"] = f"unhealthy: {e}"

# 3. Bedrock model accessible
try:
    bedrock.invoke_model(
        modelId=MODEL_ID,
        body=json.dumps({"inputText": "health check"}),
    )
    checks["bedrock"] = "healthy"
except Exception as e:
    checks["bedrock"] = f"unhealthy: {e}"

# 4. S3 bucket accessible
try:
    s3.head_bucket(Bucket=BUCKET_NAME)
    checks["s3"] = "healthy"
except Exception as e:
    checks["s3"] = f"unhealthy: {e}"

all_healthy = all(v == "healthy" for v in checks.values())

return {
    "statusCode": 200 if all_healthy else 503,
    "body": json.dumps({
        "status": "healthy" if all_healthy else "degraded",
        "region": os.environ["AWS_REGION"],
        "checks": checks,
        "timestamp": datetime.utcnow().isoformat(),
    })
}

```

Route 53 Health Check Configuration:

Parameter	Value
Protocol	HTTPS
Port	443
Path	/health
Request Interval	30 seconds
Failure Threshold	3 consecutive failures
String Matching	<code>"status": "healthy"</code>
Regions	us-east-1, us-west-2, eu-west-1 (3 health check locations minimum)

3.3 Operating Modes

Mode 1: Active-Active (Normal Operation)

- Both regions serve traffic based on geolocation
- Each region processes its own submissions independently
- DynamoDB Global Tables sync status across regions

Mode 2: Single Region / Failover

- Triggered automatically when health check fails (3 consecutive failures = 90 seconds)
- All traffic routes to the healthy region
- Recovery: when the failed region's health check passes again, traffic gradually returns

Manual Override:

- Set Route 53 record weight to 0 for a region to drain traffic (maintenance window)
 - Use Route 53 "Failover" routing policy type for strict primary/secondary if needed
-

4. User Session Stickiness

Problem

A user uploads in us-east-1, but their next request (checking status, downloading report) could route to us-west-2 where the data hasn't replicated yet.

Solution: Multi-Layer Stickiness



Implementation Details:

1. **Geolocation routing provides natural stickiness** — a user in New York always resolves to us-east-1 unless it's unhealthy. This covers 95% of cases.
2. **owner_region field in DynamoDB** — when a submission is created, stamp it with the creating region. If a user happens to hit a different region (VPN, travel, failover), the API can check this field and either:
 - Read the data (it's replicated via Global Tables, usually within 1 second)

- Return a "processing in another region" status for very recent submissions

3. **Cognito region affinity** — not strictly required if using geolocation routing, but useful as a defense-in-depth mechanism for API-level routing decisions.

5. DynamoDB Global Tables

5.1 Migration from Standard Table to Global Table

Current table: `prescoach-dev-kiro-submissions` in us-east-1

```
# CDK change: Enable Global Tables (Version 2019.11.21)
submissions_table = dynamodb.Table(
    self,
    "SubmissionsTable",
    table_name=f"{resource_prefix}-submissions",
    partition_key=dynamodb.Attribute(
        name="submission_id",
        type=dynamodb.AttributeType.STRING,
    ),
    billing_mode=dynamodb.BillingMode.PAY_PER_REQUEST,
    removal_policy=RemovalPolicy.RETAIN,
    replication_regions=["us-west-2"], # ← Add this
)
```

5.2 Replication Characteristics

Characteristic	Value
Replication Latency	Typically < 1 second (eventually consistent)
Conflict Resolution	Last-writer-wins (based on timestamp)
Write Capacity	Each region can accept writes independently

Characteristic	Value
Read Consistency	Strongly consistent reads available within the local region
Cross-Region Reads	Eventually consistent (sub-second in most cases)

5.3 Schema Changes for Multi-Region

Add these fields to the submissions table:

```
{
  "submission_id": "sub-98765",
  "user_id": "abc123",
  "owner_region": "us-east-1",
  "processing_status": "Processing",
  "created_at": "2026-06-28T14:30:00Z",
  "updated_at": "2026-06-28T14:30:05Z",
  "ttl": 1756540800
}
```

- **owner_region**: Region where the submission was created and is being processed
- **ttl**: Optional TTL for automatic cleanup of old records

5.4 Conflict Avoidance Strategy

Since submissions are created and processed in a single region, write conflicts are minimal:

- A submission is **created** in one region only
- Status updates happen **sequentially** in the creating region
- The other region only **reads** the replicated data
- Conflict scenario: Only during failover mid-processing (handled by idempotency)

5.5 Transactional Guarantees

DynamoDB Global Tables use **eventual consistency** across regions (typically sub-second). For this application:

Operation	Consistency Needed	Global Tables Behavior
Create submission	Strong (local)	✓ Strong consistent in creating region
Update status	Strong (local)	✓ Only the processing region writes
Read status (same region)	Strong	✓ Strongly consistent read available
Read status (other region)	Eventually consistent	✓ Sub-second lag acceptable for status checks
List user submissions	Eventually consistent	✓ Acceptable — user sees all recent submissions

Bottom line: No transactional writes needed across regions. Each submission is owned by one region, and Global Tables provide the read replication automatically.

6. S3 Cross-Region Replication (CRR)

6.1 What to Replicate

S3 Bucket Structure		
prescoach-dev-kiro-uploads/		
├ uploads/{user_id}/{submission_id}/{filename}	← CRR: YES	
├ transcripts/{submission_id}/transcript.txt	← CRR: YES	
├ processed/{user_id}/{sub_id}/chunks/	← CRR: NO	
├ {submission_id}/embeddings/	← CRR: NO	
├ evaluations/{submission_id}/{dimension}/	← CRR: YES	
└ reports/{user_id}/{submission_id}/	← CRR: YES	

Prefix	Replicate?	Reason
<code>uploads/</code>	Yes	Original files needed if failover occurs mid-processing
<code>transcripts/</code>	Yes	Transcripts needed for evaluation in either region
<code>processed/.../chunks/</code>	No	Ephemeral — can be regenerated from original upload
<code>{sub_id}/embeddings/</code>	No	Ephemeral — can be regenerated; large volume of small files
<code>evaluations/</code>	Yes	Final evaluation results — needed for report access in either region
<code>reports/</code>	Yes	PDF reports must be downloadable from either region

6.2 CRR Configuration

```
# CDK: S3 bucket with CRR (bidirectional)
from aws_cdk import aws_s3 as s3, aws_iam as iam

# Primary bucket (us-east-1)
uploads_bucket = s3.Bucket(
    self,
    "UploadsBucket",
    bucket_name=f"{resource_prefix}-uploads",
    versioned=True, # ← REQUIRED for CRR (change from current False)
    removal_policy=RemovalPolicy.RETAIN,
    block_public_access=s3.BlockPublicAccess.BLOCK_ALL,
    encryption=s3.BucketEncryption.S3_MANAGED,
)

# CRR rule (applied via CfnBucket ReplicationConfiguration)
# Replicate uploads/, transcripts/, evaluations/, reports/
```

Key CRR Requirements:

1. **Enable versioning** on both source and destination buckets (currently versioning is `False` — must change)
2. **Create IAM replication role** with `s3:GetReplicationConfiguration`, `s3:ReplicateObject`, `s3:ReplicateDelete`
3. **Bidirectional replication** — both buckets replicate to each other (for failover writes)
4. **Prefix filters** — only replicate the prefixes that matter (exclude chunks and embeddings)

6.3 Replication Rules

```
{
  "Rules": [
    {
      "ID": "replicate-uploads",
      "Status": "Enabled",
      "Filter": { "Prefix": "uploads/" },
      "Destination": {
        "Bucket": "arn:aws:s3:::prescoach-dev-kiro-uploads-west",
        "StorageClass": "STANDARD"
      },
      "DeleteMarkerReplication": { "Status": "Enabled" }
    },
    {
      "ID": "replicate-transcripts",
      "Status": "Enabled",
      "Filter": { "Prefix": "transcripts/" },
      "Destination": {
        "Bucket": "arn:aws:s3:::prescoach-dev-kiro-uploads-west",
        "StorageClass": "STANDARD"
      }
    },
    {
      "ID": "replicate-evaluations",
      "Status": "Enabled",
      "Filter": { "Prefix": "evaluations/" },
      "Destination": {
        "Bucket": "arn:aws:s3:::prescoach-dev-kiro-uploads-west",
        "StorageClass": "STANDARD"
      }
    }
  ]
}
```

```

    }
  },
  {
    "ID": "replicate-reports",
    "Status": "Enabled",
    "Filter": { "Prefix": "reports/" },
    "Destination": {
      "Bucket": "arn:aws:s3:::prescoach-dev-kiro-uploads-west",
      "StorageClass": "STANDARD"
    }
  }
]
}

```

6.4 S3 Bucket Naming Strategy

Since S3 bucket names are globally unique, use a region suffix:

Region	Bucket Name
us-east-1	<code>prescoach-dev-kiro-uploads</code> (existing, keep as-is)
us-west-2	<code>prescoach-dev-kiro-uploads-west</code>

Application code reads the bucket name from environment variable `S3_BUCKET_NAME` — no code changes needed.

6.5 Replication Timing

Object Size	Typical Replication Time
< 1 MB (transcripts, JSON)	< 15 seconds
1-50 MB (audio files)	15-60 seconds
50-500 MB (large recordings)	1-5 minutes

Object Size	Typical Replication Time
PDF reports (~200 KB)	< 15 seconds

S3 Replication Time Control (RTC) — optional, guarantees 99.99% of objects replicate within 15 minutes. Adds cost but provides SLA for compliance.

7. Cognito Multi-Region Strategy

7.1 Challenge

Cognito User Pools are regional and **do not natively replicate**. Options:

Approach	Pros	Cons
Single Cognito in us-east-1	Simple, no sync issues	Cross-region auth latency, single point of failure
Cognito per region + custom sync	Low auth latency	Complex user sync, conflict resolution
Cognito per region + shared backend DB	Independent auth	Token validation works cross-region with JWKS

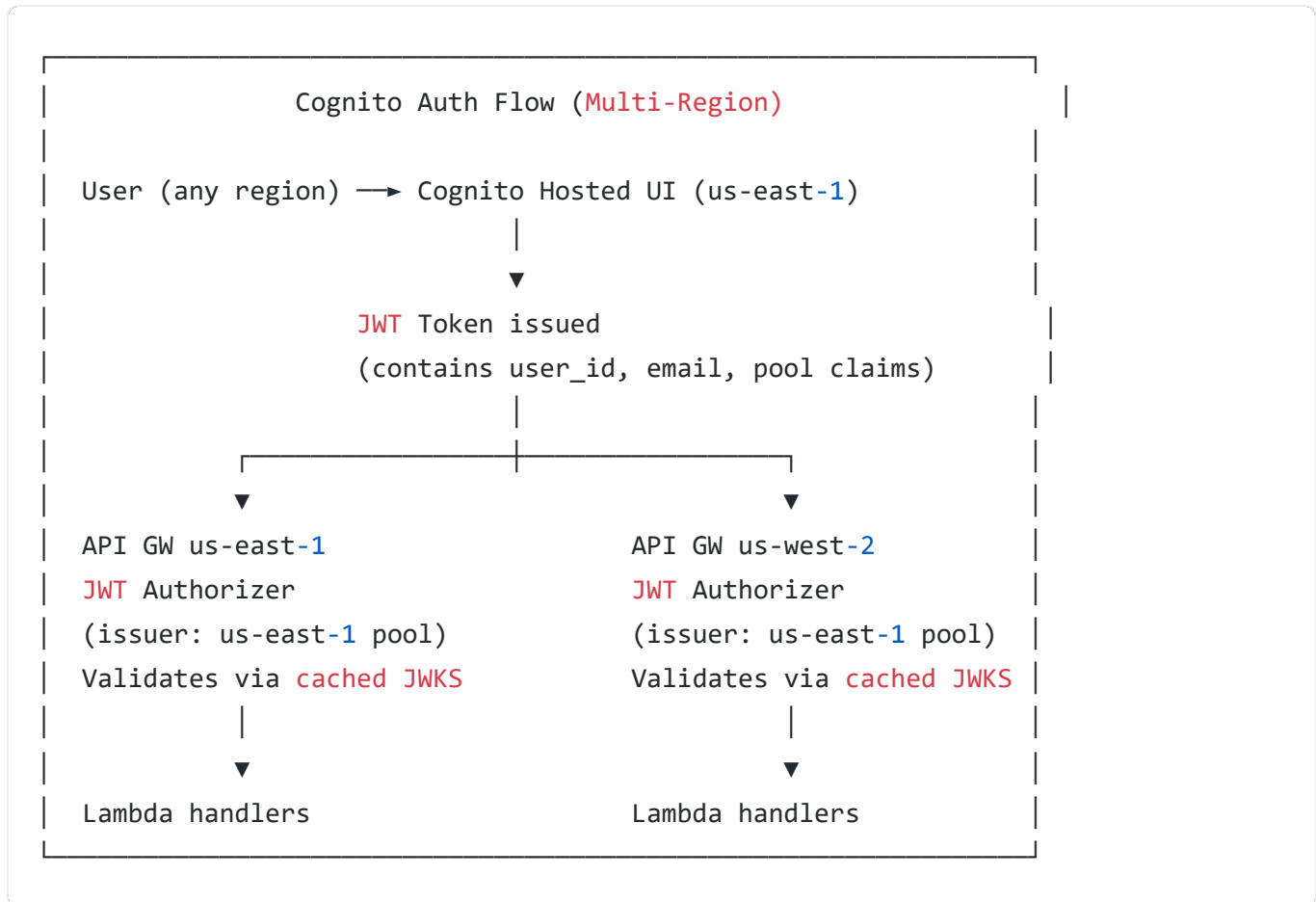
7.2 Recommended: Single Cognito with Regional Token Validation

Keep **one Cognito User Pool in us-east-1** (current setup). Both regions validate JWT tokens using the JWKS endpoint:

```
JWKS URL: https://cognito-idp.us-east-1.amazonaws.com/{pool_id}/.well-known/jwks.json
```

- API Gateway JWT authorizer in us-west-2 points to the us-east-1 Cognito issuer

- Tokens are validated locally using cached JWKS keys (no cross-region call per request)
- Sign-up/sign-in goes to us-east-1 Cognito (acceptable — auth is infrequent)
- Token refresh uses us-east-1 (30-day refresh token, 1-hour access token)



Tradeoff: Authentication (sign-in) has slightly higher latency for us-west-2 users since Cognito is in us-east-1. But sign-in is a one-time event per session (1-hour token, 30-day refresh). API request authorization is fast in both regions via cached JWKS validation.

Failover consideration: If us-east-1 is completely down, users cannot sign in or refresh tokens. Mitigation: users with valid tokens (up to 1 hour old) continue working. For true auth HA, implement a backup Cognito pool in us-west-2 with user data sync via Lambda triggers — this is a Phase 2 enhancement.

8. Service-by-Service Multi-Region Design

8.1 API Gateway

Component	Multi-Region Approach
HTTP API	Deploy identical API in both regions
Custom Domain	<code>api.prescoach.geiseraai.com</code> with regional API mapping
JWT Authorizer	Both point to us-east-1 Cognito issuer
CORS	Both allow <code>https://kiro.geiseraai.com</code>

8.2 Lambda Functions

- Deploy identical function code in both regions
- Each function reads regional environment variables (bucket name, table name, queue URL)
- No cross-region calls from Lambda — everything is regional

8.3 Step Functions

- Deploy identical state machine in both regions
- Each region's workflow processes only submissions created in that region
- SSM parameters are regional (same keys, same values, separate stores)

8.4 SQS Queues

- SQS is regional — deploy identical queue structure in both regions
- Messages are NOT replicated cross-region (no need — processing is regional)
- Queue names remain the same in both regions

8.5 ECS Fargate (Evaluation)

- Deploy identical ECS cluster + task definition in both regions
- Each region's evaluator processes only its own handoff queue
- Bedrock calls use the local region (no cross-region model invocation needed)

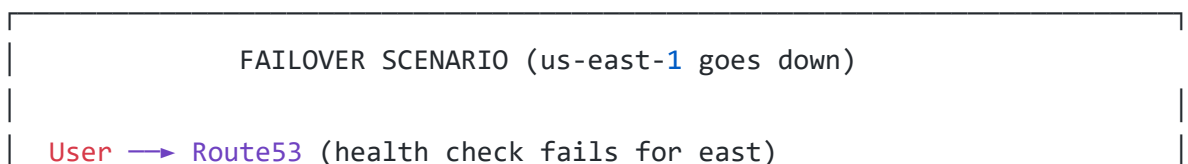
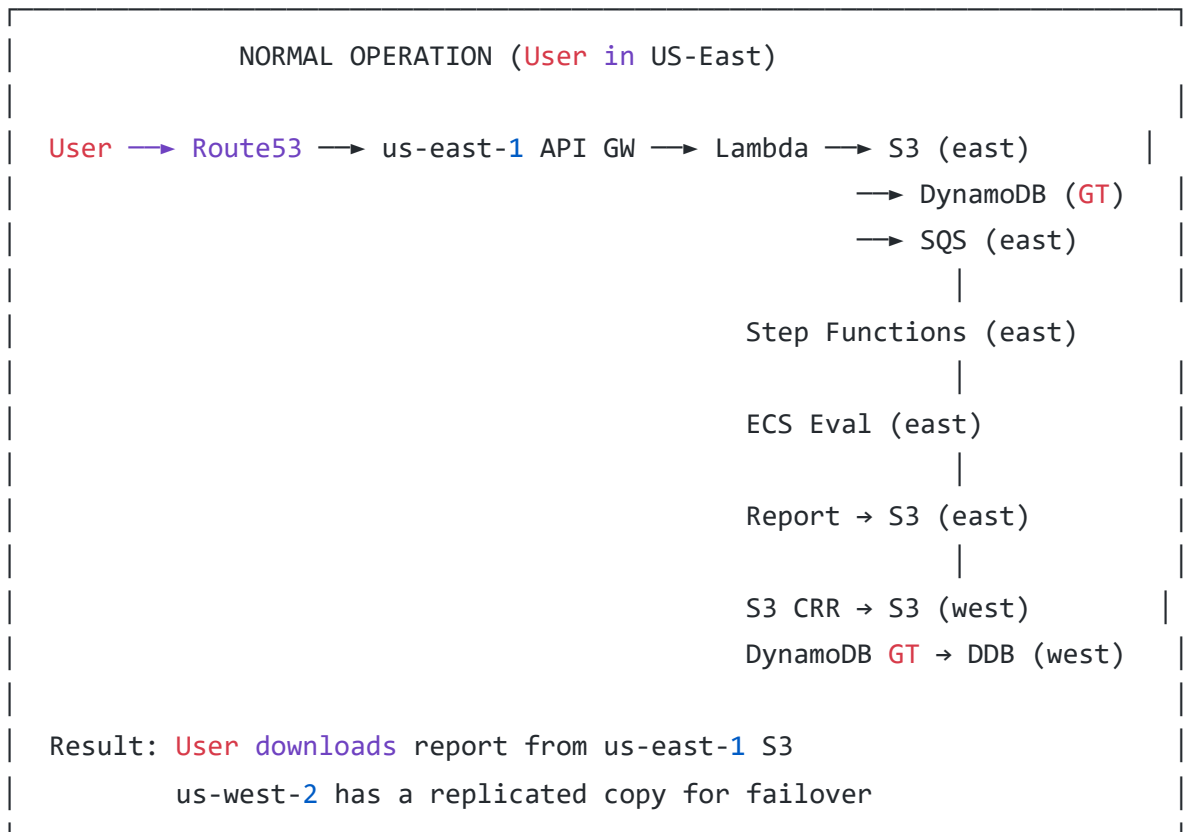
8.6 SNS Topics

- Regional topics for error notifications in each region
- Optional: cross-region subscription for centralized alerting (SNS supports cross-region subscriptions)

8.7 CloudFront (Webapp)

- Single CloudFront distribution with **multiple origins** (S3 bucket per region)
- Origin failover group: primary = nearest region, secondary = other region
- Or: separate distributions per region with Route 53 geolocation routing

9. Data Flow in Multi-Region



```

    → us-west-2 API GW → Lambda → S3 (west, has replicated)
                                   → DynamoDB (GT, replicated)

New submissions: processed entirely in us-west-2
In-flight (east): lost mid-processing, DDB shows "Processing"
└─ Recovery: resubmit from S3 (original upload is replicated)

User downloads existing reports from us-west-2 S3 (CRR copy)

```

10. CDK Multi-Region Deployment Strategy

10.1 Stack Organization

```

kiro-agentcore-app/
├─ infra/
│   ├── app.py                # CDK App entry point
│   ├── multi_region_config.py # Region-specific configuration
│   ├── global_stack.py       # Route 53, health checks (deploy once)
│   ├── regional_stack.py     # Per-region resources (deploy in each region)
│   └─ replication_stack.py   # S3 CRR rules, DynamoDB GT config

```

10.2 Deployment Order

Phase 1: Deploy Regional Stacks (parallel)

```

├─ cdk deploy RegionalStack-us-east-1 --region us-east-1
└─ cdk deploy RegionalStack-us-west-2 --region us-west-2

```

Phase 2: Enable Replication

```

├─ DynamoDB Global Table replica (add us-west-2)
└─ S3 CRR rules (bidirectional)

```

Phase 3: Deploy Global Stack

```

└─ Route 53 records + health checks (region-agnostic)

```

Phase 4: DNS Cutover

└─ Update `api.prescoach.geiseraai.com` to use new geolocation records

10.3 Environment Variables Per Region

Variable	us-east-1	us-west-2
<code>S3_BUCKET_NAME</code>	<code>prescoach-dev-kiro-uploads</code>	<code>prescoach-dev-kiro-uploads-west</code>
<code>DYNAMODB_TABLE_NAME</code>	<code>prescoach-dev-kiro-submissions</code>	<code>prescoach-dev-kiro-submissions</code> (same — Global Table)
<code>SQS_QUEUE_URL</code>	<code>https://sqs.us-east-1.amazonaws.com/...</code>	<code>https://sqs.us-west-2.amazonaws.com/...</code>
<code>AWS_DEFAULT_REGION</code>	<code>us-east-1</code>	<code>us-west-2</code>
<code>COGNITO_ISSUER</code>	<code>https://cognito-idp.us-east-1.amazonaws.com/{pool_id}</code>	Same (shared Cognito)
<code>OWNER_REGION</code>	<code>us-east-1</code>	<code>us-west-2</code>

11. Additional Concerns and Considerations

11.1 Bedrock Model Access

- Request model access in **both regions** via the Bedrock console
- Cross-region inference (`us.` prefix) already handles this — but verify model availability in us-west-2
- Quota limits are **per-region** — you get separate token-per-minute quotas in each region (benefit: 2x total capacity)

11.2 Cost Implications

Component	Additional Cost (Multi-Region)		
DynamoDB Global Tables	~35% more than single-region (replicated write capacity)		
S3 CRR	Data transfer out (\$0.02/GB) + PUT request cost in destination		
Lambda (duplicate)	Pay only for invocations — no idle cost		
ECS Fargate Spot	Second cluster — but desired_count=0, pay only when processing		
SQS	Negligible (per-request pricing)		
Route 53	\$1.50/month for health checks + \$0.50/hosted zone		
	API Gateway	Pay per request — no idle cost	
	CloudFront	Shared globally already	
	Estimated overhead	**\$5-15/month at low volume** (dominated by DynamoDB GT + S3 transfer)	

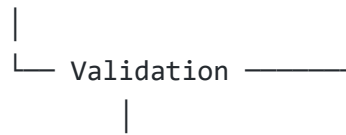
11.3 CI/CD Pipeline Updates

Current: Single-region CodePipeline

└ Build → Test → Deploy (us-east-1)

Multi-Region: Parallel deployment

└ Build → Test → Deploy us-east-1 → Deploy us-west-2



Route 53 health **check** confirms both regions healthy

Options:

1. **Single pipeline with cross-region deploy stage** — CodePipeline can deploy to other regions
2. **Separate pipelines per region** triggered by the same source — simpler isolation
3. **CDK Pipelines with cross-region support** — CDK has native multi-region deployment

Recommended: **CDK Pipelines** — it natively supports deploying stacks to multiple regions in the correct order.

11.4 In-Flight Submissions During Failover

Problem: If us-east-1 goes down while a submission is mid-processing (status = "Processing" or "Evaluating"), that work is lost.

Recovery Strategy:

1. DynamoDB Global Table has the record with status "Processing" or "Evaluating"
2. S3 CRR has already replicated the original upload file
3. A **recovery Lambda** in the healthy region scans for stale "Processing" records:
 - `processing_status IN ("Processing", "Evaluating") AND owner_region = "us-east-1" AND updated_at < (now - 30 minutes)`
4. Re-queue those submissions into the local SQS queue for reprocessing
5. Step Functions is idempotent — reprocessing produces the same result

11.5 SQS Message Loss During Failover

SQS is regional and does NOT replicate. Messages in the failed region's queue are lost if the region is down.

Mitigation:

- Messages on the handoff FIFO queue represent work-in-progress
- The DynamoDB record (replicated) tracks what was queued
- Recovery Lambda identifies submissions stuck in "Waiting" status and re-queues them

11.6 Amazon Transcribe Regional Behavior

- Transcribe jobs are regional (same region as the S3 source file)
- In failover: the replicated upload file is transcribed in the new region
- No cross-region Transcribe call needed

11.7 SSM Parameter Store

- SSM is regional — deploy identical parameters in both regions
- Use CDK to ensure consistency (same parameter values deployed to both)
- Feature flags (like `embeddings-enabled`) must be toggled in both regions

11.8 Monitoring and Observability

Add a **cross-region dashboard** in CloudWatch:

```
CloudWatch Dashboard: "PresCoach-MultiRegion"
├─ Widget: API Latency (us-east-1 vs us-west-2)
├─ Widget: Active Submissions per Region
├─ Widget: DynamoDB Replication Lag
├─ Widget: S3 CRR Pending Replication Count
├─ Widget: Health Check Status (both regions)
├─ Widget: Bedrock Throttling per Region
└─ Widget: ECS Task Count per Region
```

S3 Replication Metrics to Monitor:

- `ReplicationLatency` — time to replicate objects
- `OperationsPendingReplication` — backlog size
- `OperationsFailedReplication` — failures (alert on > 0)

11.9 Data Sovereignty / Compliance

- If users are only in the US, us-east-1 + us-west-2 keeps all data within US boundaries
- Audio recordings may contain PII (voices, names) — both regions maintain the same encryption posture
- HIPAA/SOC2: DynamoDB Global Tables and S3 CRR are both HIPAA eligible

11.10 Testing the Multi-Region Setup

Test	Method	Expected Result
Geographic routing	<code>dig api.prescoach.geiserai.com</code> from different regions	Returns regional IP
Health check failover	Stop health check Lambda in one region	Traffic shifts within 90s
DynamoDB replication	Write in east, read in west	Record appears in < 2s
S3 CRR	Upload in east, check west	Object appears within replication window
In-flight recovery	Kill ECS task mid-evaluation, check recovery	Re-queued within 30 min scan
Auth cross-region	Sign in, then hit us-west-2 API	JWT validates successfully
Report download from secondary	Upload in east, download from west (after CRR)	PDF downloads correctly

12. Implementation Phases

Phase 1: Foundation (Week 1-2)

- ☐ Enable S3 versioning on existing bucket (required for CRR)
- ☐ Add `owner_region` field to DynamoDB writes
- ☐ Create health check Lambda function
- ☐ Parameterize region-specific values in CDK (bucket names, queue URLs)
- ☐ Validate Bedrock model access in us-west-2

Phase 2: Second Region Deploy (Week 3-4)

- ☐ Deploy all stacks to us-west-2 (new S3 bucket, queues, Lambda, Step Functions, ECS)
- ☐ Convert DynamoDB to Global Table (add us-west-2 replica)
- ☐ Configure S3 CRR (bidirectional, filtered by prefix)
- ☐ Deploy health check endpoints in both regions
- ☐ Test end-to-end processing in us-west-2 independently

Phase 3: Route 53 and Traffic Management (Week 5)

- ☐ Create Route 53 geolocation records
- ☐ Configure health checks with string matching
- ☐ Set up failover behavior
- ☐ Update webapp to use the new domain (if not already using custom domain)
- ☐ Test geographic routing with VPN/proxy

Phase 4: Failover Testing and Hardening (Week 6)

- ☐ Simulate us-east-1 failure (disable health check)
- ☐ Verify traffic shifts to us-west-2
- ☐ Test in-flight submission recovery
- ☐ Build cross-region CloudWatch dashboard
- ☐ Document runbook for manual failover/recovery
- ☐ Load test both regions simultaneously

Phase 5: CI/CD Multi-Region (Week 7)

- ☐ Update CodePipeline to deploy to both regions

- ☐ Add post-deploy health validation
- ☐ Implement blue/green deployment strategy per region
- ☐ Create rollback procedures

13. Decision Summary

Decision	Choice	Rationale
Regions	us-east-1 + us-west-2	Full Bedrock parity, geographic coverage, no pricing difference
Routing	Geolocation + health checks	Natural stickiness, automatic failover
DynamoDB	Global Tables	Sub-second replication, no code changes for reads
S3	Bidirectional CRR (filtered)	Only replicate final artifacts, not ephemeral data
Cognito	Single pool (us-east-1)	Simplicity; token validation is regional via JWKS caching
SQS	Regional (no replication)	Processing is regional; recovery via DynamoDB scan
Processing model	Each region is self-contained	No cross-region service calls during normal operation
Failover detection	Route 53 health checks (30s interval, 3 failures)	90-second detection time

14. Risks and Mitigations

Risk	Impact	Mitigation
DynamoDB replication lag during writes	Stale status reads from secondary region	Application tolerates eventual consistency for status checks
S3 CRR delay for large audio files	Report not immediately available in secondary region	Show "replicating" status; RTC option for guaranteed 15-min SLA
Cognito single point of failure	Cannot sign in if us-east-1 is down	1-hour token validity provides buffer; Phase 2: backup Cognito pool
In-flight submissions lost during failover	Submissions stuck in "Processing"	Recovery Lambda rescans and re-queues after timeout
Cost increase	~\$5-15/month overhead at low volume	Acceptable for HA; dominated by DynamoDB GT replication writes
Deployment complexity	More moving parts, more failure modes	CDK Pipelines automates; infrastructure tests validate both regions
Split-brain during network partition	Both regions accept writes for same submission	Prevented by owner_region field + last-writer-wins resolution

Document created: June 28, 2026 Architecture: Presentation Coaching Platform — Multi-Region Active-Active Regions: us-east-1 (primary) + us-west-2 (secondary)